Algorithm: Report Management System

1.Start the program.

2.Define three decorators: uppercase, bold, and add_border.

3.Create a class Report.

4.Create a class variable templates to store report templates.

5.Define a constructor to initialize title, author, and sections.

6.Define add_template() to save a template.

7.Define create_from_template() to create a report using a template.

8.Define add_section() to add a section to the report.

9.Define fill_section() to update section content.

10.Define summary() and apply decorators to format the summary.

11.Define __str__() to display the report.

12.Define __len__() to return the number of sections.

13.Define __getitem__() to access sections by index.

14.Define __add__() to combine two reports.

15.Define __eq__() to compare two reports.

16.Create a report template named project_report.

17.Create report r1 from the template.

18.Fill the sections of r1.

19.Create another report r2.

20.Add a section to r2.

21.Display r1.

22.Display the number of sections in r1.

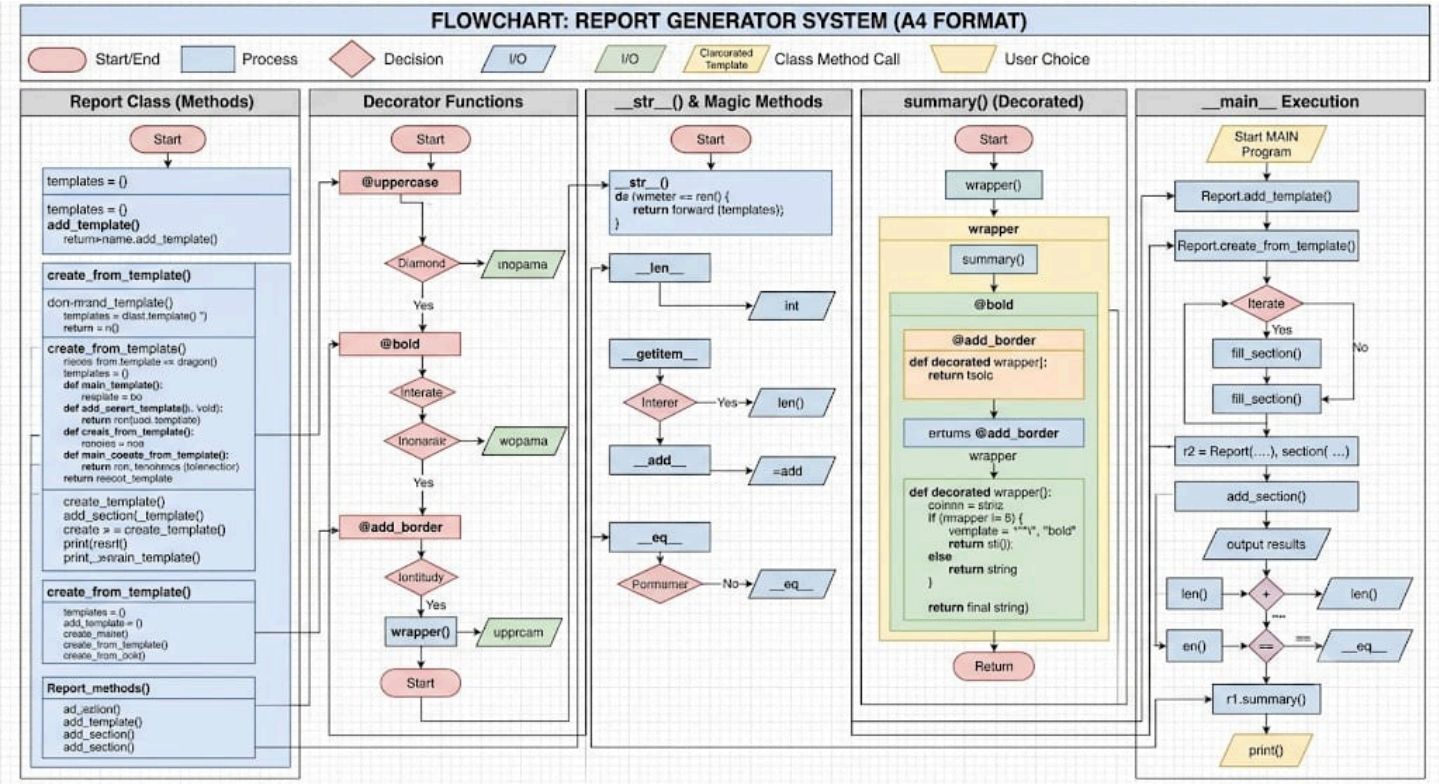
23.Display the first section of r1.

24.Combine r1 and r2 and display the combined report.

25.Compare reports and display the result.

26.Display the formatted summary of r1.

27.End the program.



Strategy Pattern Implementaion - Python Source Code

```
def uppercase(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper



def bold(func):
    def wrapper(*args, **kwargs):
        return "**" + func(*args, **kwargs) + "**"
    return wrapper
```

```
def add_border(func):
    def wrapper(*args, **kwargs):
        border = "-" * 40
        return border + "\n" + func(*args, **kwargs) + "\n" + border
    return wrapper



class Report:
    templates = {}

    def __init__(self, title, author="Unknown"):
        self.title = title
        self.author = author
        self.sections = []

    @classmethod
    def add_template(cls, name, section_list):
        cls.templates[name] = section_list
        print(f"Template '{name}' saved with sections: {section_list}")

    @classmethod
    def create_from_template(cls, template_name, title, author="Unknown"):
        new_report = cls(title, author)
        for heading in cls.templates[template_name]:
```

```
        new_report.add_section(heading, "Content not filled yet")
    return new_report

def add_section(self, heading, content):
    self.sections.append((heading, content))

def fill_section(self, heading, content):
    for i in range(len(self.sections)):
        if self.sections[i][0] == heading:
            self.sections[i] = (heading, content)
            return True
    return False

@bold
@add_border
def summary(self):
    return (
        f"Report: {self.title} | "
        f"Author: {self.author} | "
        f"Sections: {len(self.sections)}"
    )

def __str__(self):
    text = f"REPORT: {self.title} (by {self.author})\n"
    for heading, content in self.sections:
        text += f"- {heading}: {content}\n"
```

```
        return text

    def __len__(self):
        return len(self.sections)

    def __getitem__(self, index):
        return self.sections[index]

    def __add__(self, other):
        combined = Report(self.title + " + " + other.title, self.author)
        combined.sections = self.sections + other.sections
        return combined

    def __eq__(self, other):
        return (
            self.title == other.title
            and self.sections == other.sections
        )


if __name__ == "__main__":

    # Create Template
    Report.add_template(
        "project_report",
        ["Introduction", "Result", "Conclusion"]
```

```
)

r1 = Report.create_from_template(
    "project_report",
    "My Mini Project",
    "Ganesh"
)

r1.fill_section(
    "Introduction",
    "This project demonstrates OOP concepts in Python."
)
r1.fill_section(
    "Result",
    "The program executed successfully."
)
r1.fill_section(
    "Conclusion",
    "Decorators and magic methods improve flexibility."
)

r2 = Report("Attendance Report", "Ganesh")
r2.add_section(
    "Summary",
    "79% attendance this month."
)
```

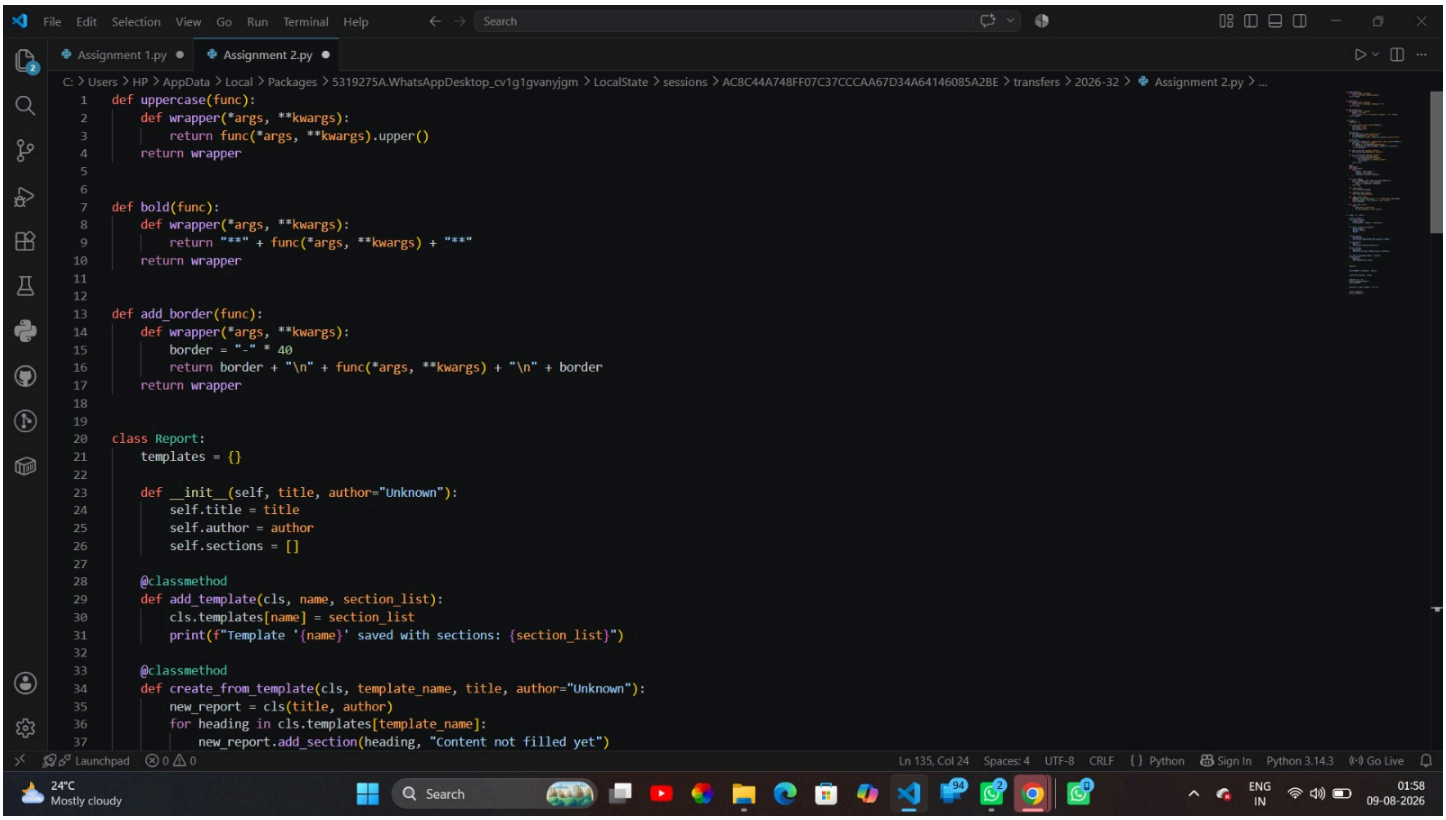
```
print(r1)


print("Number of Sections:", len(r1))


print("First Section:", r1[0])


combined = r1 + r2
print("\nCombined Report:")
print(combined)


print("Are r1 and r1 equal?", r1 == r1)
print("\nSummary:")
print(r1.summary())
```



```python
def uppercase(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper


def bold(func):
    def wrapper(*args, **kwargs):
        return "**" + func(*args, **kwargs) + "**"
    return wrapper

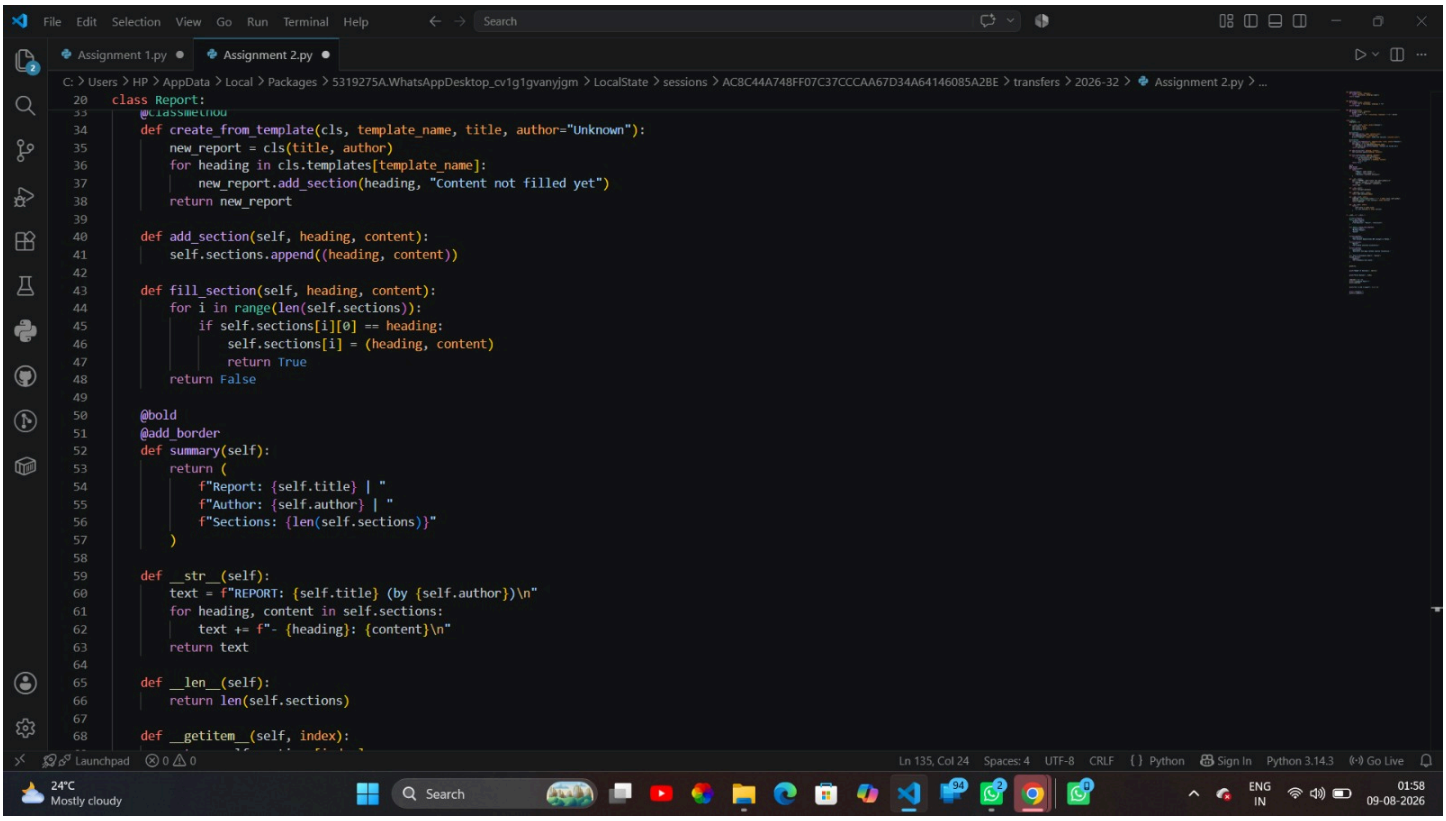

def add_border(func):
    def wrapper(*args, **kwargs):
        border = "-" * 40
        return border + "\n" + func(*args, **kwargs) + "\n" + border
    return wrapper


class Report:
    templates = {}

    def __init__(self, title, author="Unknown"):
        self.title = title
        self.author = author
        self.sections = []

    @classmethod
    def add_template(cls, name, section_list):
        cls.templates[name] = section_list
        print(f"Template '{name}' saved with sections: {section_list}")

    @classmethod
    def create_from_template(cls, template_name, title, author="Unknown"):
        new_report = cls(title, author)
        for heading in cls.templates[template_name]:
            new_report.add_section(heading, "Content not filled yet")
```



```python
class Report:
    @classmethod
    def create_from_template(cls, template_name, title, author="Unknown"):
        new_report = cls(title, author)
        for heading in cls.templates[template_name]:
            new_report.add_section(heading, "Content not filled yet")
        return new_report

    def add_section(self, heading, content):
        self.sections.append((heading, content))

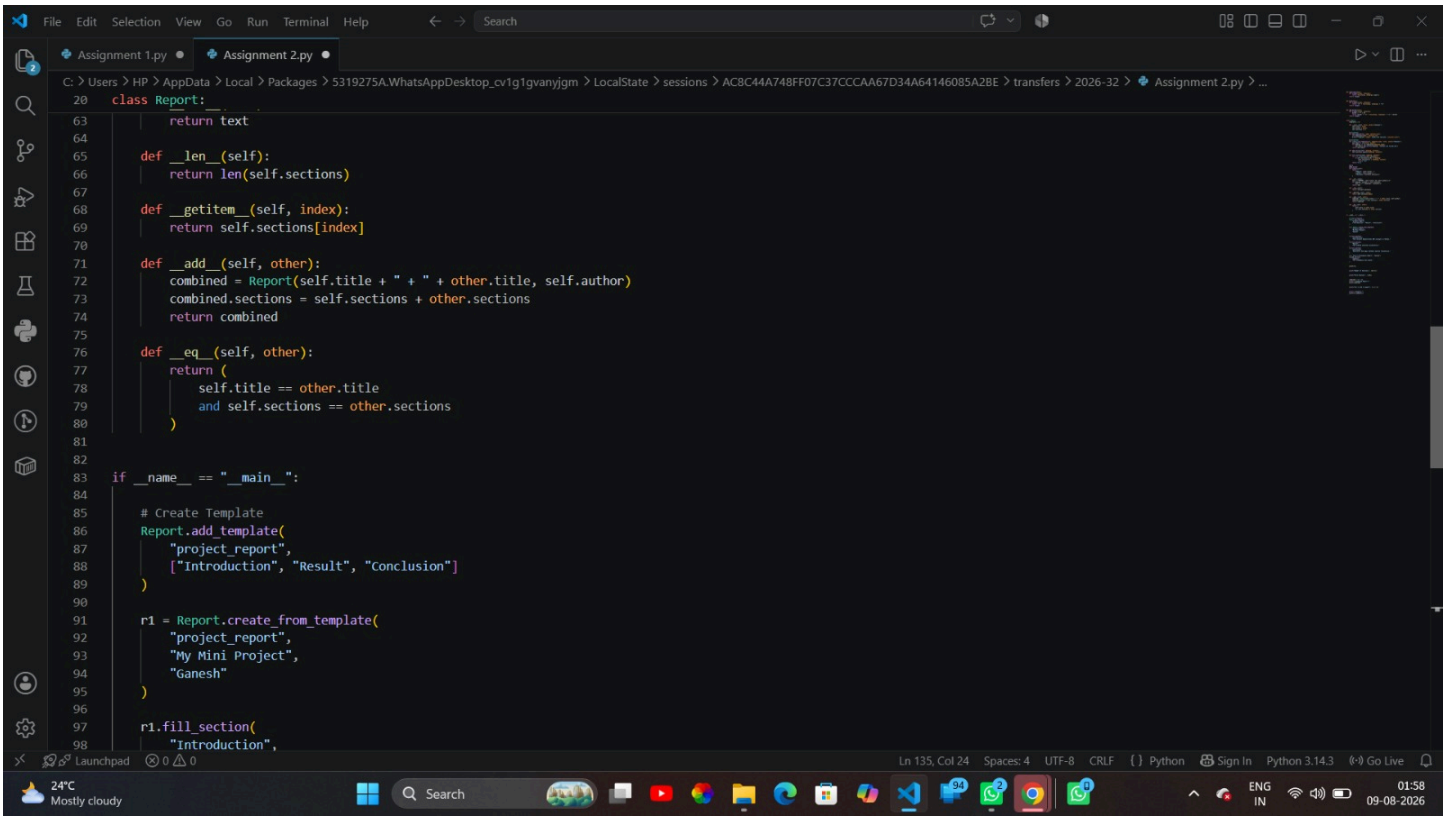
    def fill_section(self, heading, content):
        for i in range(len(self.sections)):
            if self.sections[i][0] == heading:
                self.sections[i] = (heading, content)
                return True
        return False

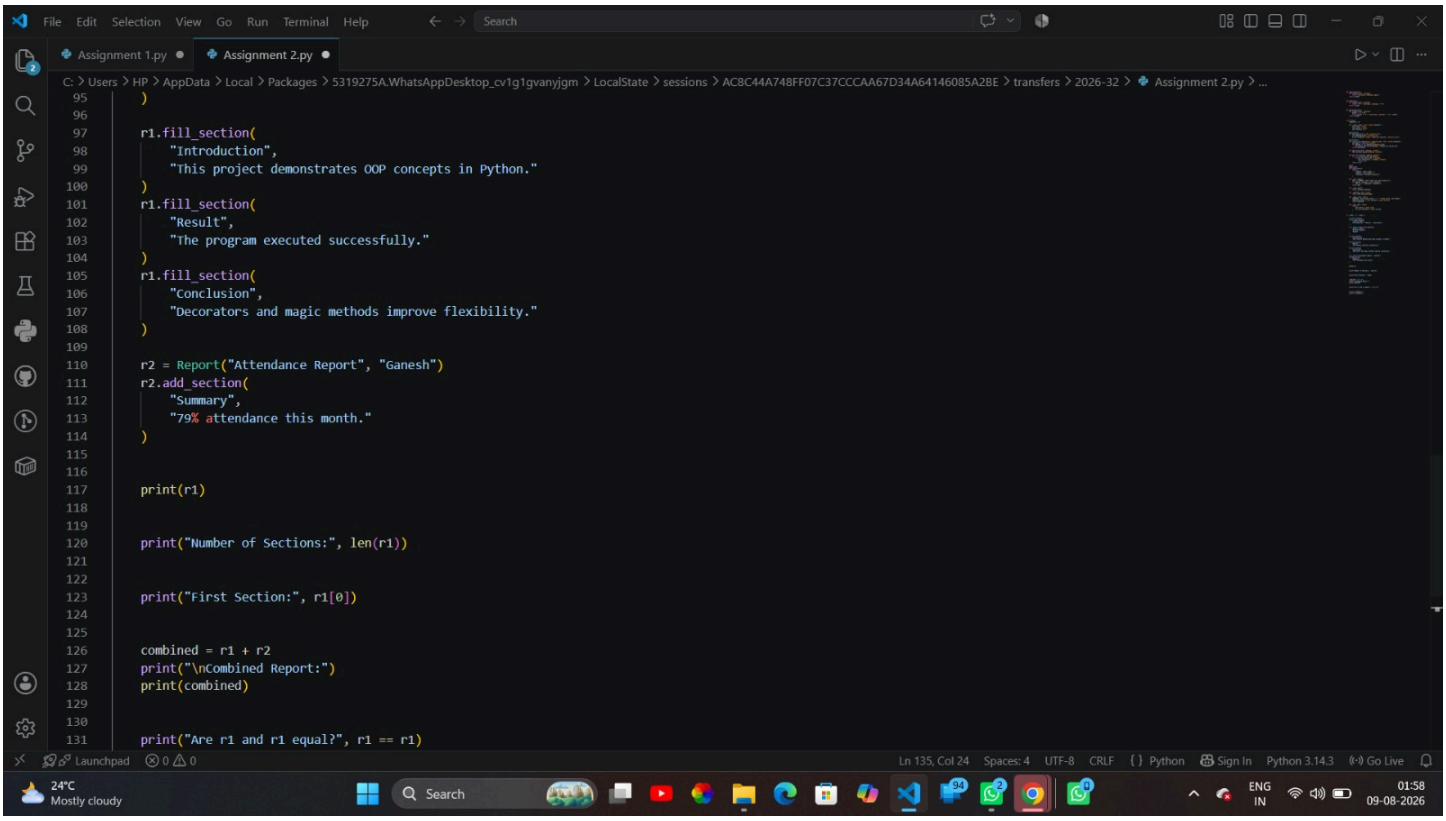
    @bold
    @add_border
    def summary(self):
        return (
            f"Report: {self.title} | "
            f"Author: {self.author} | "
            f"Sections: {len(self.sections)}"
        )

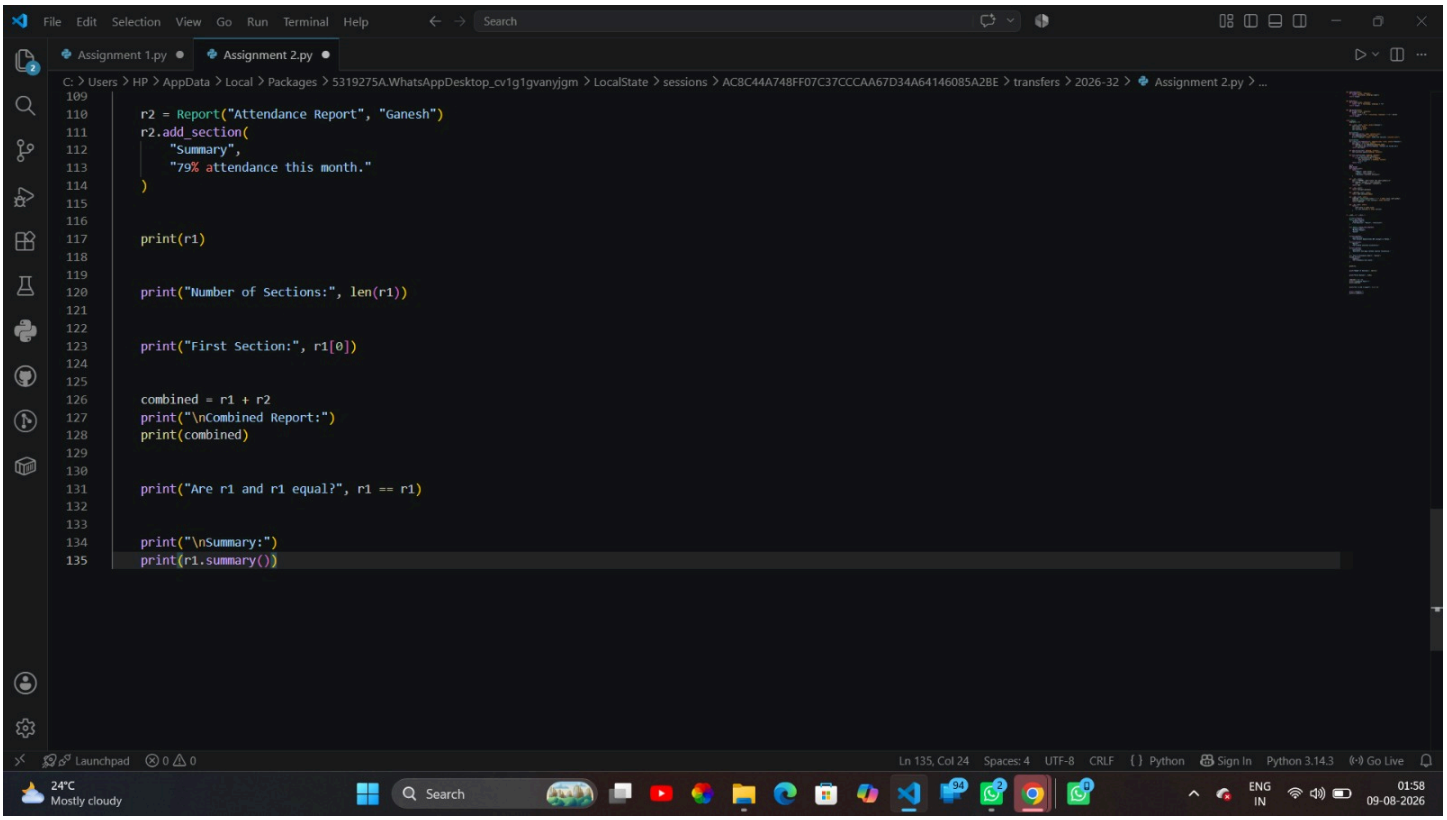
    def __str__(self):
        text = f"REPORT: {self.title} (by {self.author})\n"
        for heading, content in self.sections:
            text += f"- {heading}: {content}\n"
        return text

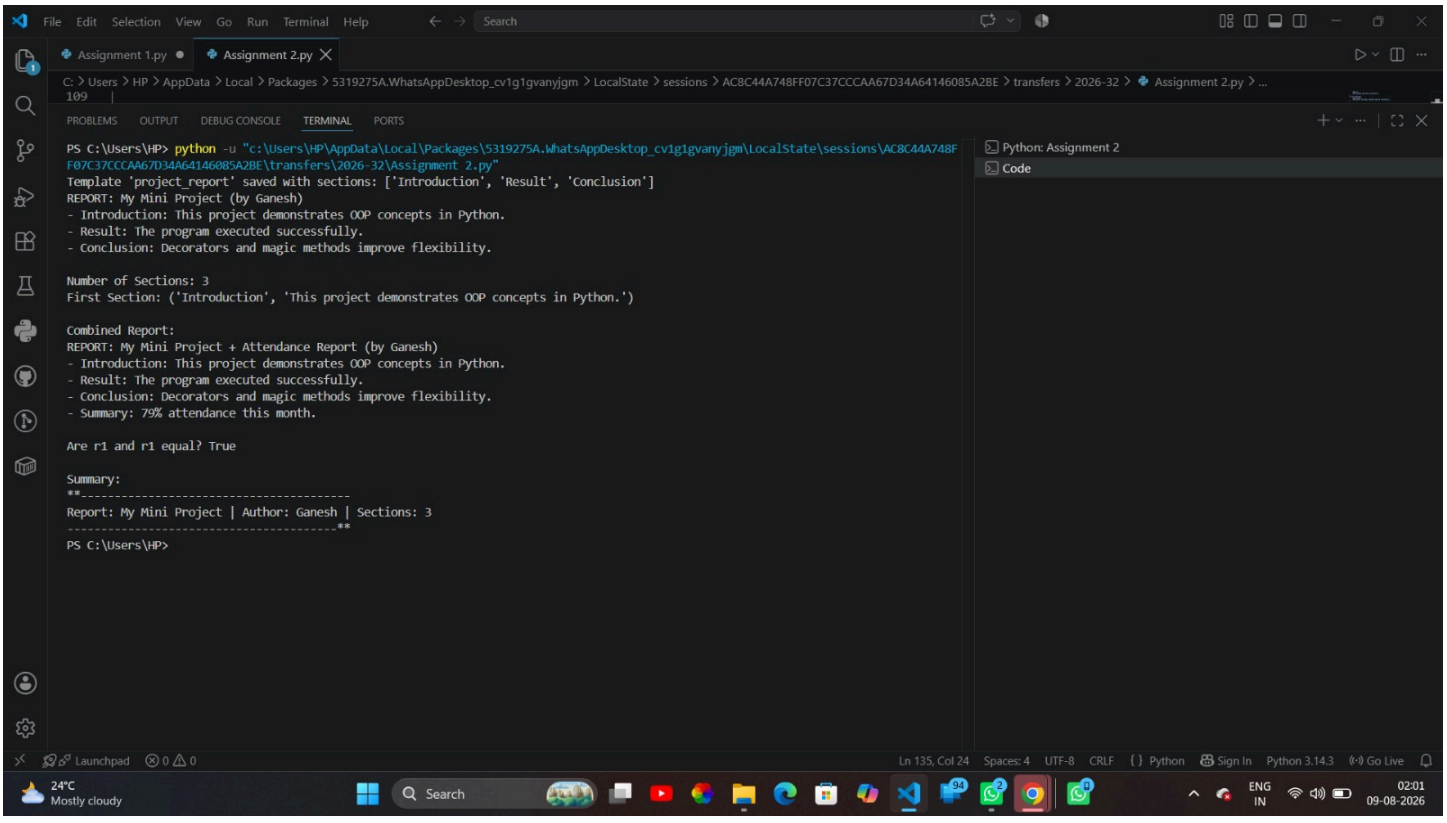
    def __len__(self):
        return len(self.sections)

    def __getitem__(self, index):
```


File Edit Selection View Go Run Terminal Help
Search
Assignment 1.py
Assignment 2.py
C: > Users > HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-32 > Assignment 2.py > ...
20 class Report:
63 return text
64
65 def __len__(self):
66 return len(self.sections)
67
68 def __getitem__(self, index):
69 return self.sections[index]
70
71 def __add__(self, other):
72 combined = Report(self.title + " + " + other.title, self.author)
73 combined.sections = self.sections + other.sections
74 return combined
75
76 def __eq__(self, other):
77 return (
78 self.title == other.title
79 and self.sections == other.sections
80)
81
82
83 if __name__ == "__main__":
84
85 # Create Template
86 Report.add_template(
87 "project_report",
88 ["Introduction", "Result", "Conclusion"]
89)
90
91 r1 = Report.create_from_template(
92 "project_report",
93 "My Mini Project",
94 "Ganesh"
95)
96
97 r1.fill_section(
98 "Introduction",
Launchpad
Ln 135, Col 24 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14.3 Go Live
24°C Mostly cloudy
Search
ENG IN
01:58 09-08-2026


File Edit Selection View Go Run Terminal Help
Search
Assignment 1.py
Assignment 2.py
C: > Users > HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-32 > Assignment 2.py > ...
95)
96
97 r1.fill_section(
98 "Introduction",
99 "This project demonstrates OOP concepts in Python."
100)
101 r1.fill_section(
102 "Result",
103 "The program executed successfully."
104)
105 r1.fill_section(
106 "Conclusion",
107 "Decorators and magic methods improve flexibility."
108)
109
110 r2 = Report("Attendance Report", "Ganesh")
111 r2.add_section(
112 "Summary",
113 "79% attendance this month."
114)
115
116
117 print(r1)
118
119
120 print("Number of Sections:", len(r1))
121
122
123 print("First Section:", r1[0])
124
125
126 combined = r1 + r2
127 print("\nCombined Report:")
128 print(combined)
129
130
131 print("Are r1 and r1 equal?", r1 == r1)
Launchpad
Ln 135, Col 24 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14.3 Go Live
24°C Mostly cloudy
Search
ENG IN
01:58 09-08-2026


File Edit Selection View Go Run Terminal Help
Search
Assignment 1.py
Assignment 2.py
C: > Users > HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-32 > Assignment 2.py > ...
109
110 r2 = Report("Attendance Report", "Ganesh")
111 r2.add_section(
112 "Summary",
113 "79% attendance this month."
114)
115
116
117 print(r1)
118
119
120 print("Number of Sections:", len(r1))
121
122
123 print("First Section:", r1[0])
124
125
126 combined = r1 + r2
127 print("\nCombined Report:")
128 print(combined)
129
130
131 print("Are r1 and r1 equal?", r1 == r1)
132
133
134 print("\nSummary:")
135 print(r1.summary())
Launchpad
Ln 135, Col 24
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.3
Go Live
24°C
Mostly cloudy
Search
ENG
IN
01:58
09-08-2026


File Edit Selection View Go Run Terminal Help
Search
Assignment 1.py
Assignment 2.py
C: > Users > HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-32 > Assignment 2.py > ...
109
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\HP> python -u "c:\Users\HP\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\AC8C44A748FF07C37CCCAA67D34A64146085A2BE\transfers\2026-32\Assignment 2.py"
Template 'project_report' saved with sections: ['Introduction', 'Result', 'Conclusion']
REPORT: My Mini Project (by Ganesh)
- Introduction: This project demonstrates OOP concepts in Python.
- Result: The program executed successfully.
- Conclusion: Decorators and magic methods improve flexibility.
Number of Sections: 3
First Section: ('Introduction', 'This project demonstrates OOP concepts in Python.')
Combined Report:
REPORT: My Mini Project + Attendance Report (by Ganesh)
- Introduction: This project demonstrates OOP concepts in Python.
- Result: The program executed successfully.
- Conclusion: Decorators and magic methods improve flexibility.
- Summary: 79% attendance this month.
Are r1 and r1 equal? True
Summary:
**--
Report: My Mini Project | Author: Ganesh | Sections: 3
--**
PS C:\Users\HP>
Python: Assignment 2
Code
Launchpad
Ln 135, Col 24
Spaces: 4
UTF-8
CRLF
Python
Sign In
Python 3.14.3
Go Live
24°C
Mostly cloudy
Search
ENG
IN
02:01
09-08-2026